—---------– CONSIGNES —----------

Ce projet individualisé vise à développer un algorithme de mise en page automatique de pipelines acycliques (DAG) afin de produire un schéma graphique lisible à partir d'une description structurelle du pipeline. Chaque brique du pipeline correspond à un bloc rectangulaire de dimension similaire, que l'on souhaite placer sur une grille régulière. Les connexions entre blocs seront représentées par des « tuyaux » orthogonaux, c'est-à-dire composés uniquement de segments horizontaux et verticaux.

Objectifs
Les objectifs du projet sont les suivants :
— modéliser un pipeline comme un graphe orienté acyclique ;
— concevoir un algorithme de placement automatique de blocs sur une grille ;
— concevoir un algorithme de routage orthogonal des connexions entre blocs ;
— réduire, par des heuristiques, le nombre de croisements, la longueur et le nombre de coudes des connexions ;
— produire une sortie exploitable pour générer un schéma (p. ex. SVG) et évaluer la qualité des mises en page obtenues.

Contexte
Les outils de construction de pipelines permettent d'assembler rapidement des chaînes de traitement, mais la production de schémas clairs et esthétiques demeure souvent manuelle. Automatiser la mise en page de tels diagrammes est un problème algorithmique classique, combinant placement de nœuds et routage d'arêtes sous contraintes géométriques. Le cadre de ce projet se limite à des pipelines sans cycles et à des blocs de dimension similaire, ce qui permet une modélisation sur grille et des heuristiques efficaces.

Chronologie
Le projet se déroule sur une période d'environ 180 heures. Une première phase sera consacrée à l'étude du problème et à la définition d'une représentation interne (graphe, grille, contraintes). Une seconde phase portera sur l'implémentation d'un algorithme de placement (par niveaux/topologie) et d'un algorithme de routage orthogonal. Finalement, une phase d'évaluation permettra de tester l'approche sur différents pipelines, d'analyser des métriques de qualité (croisements, coudes, longueur) et de documenter les résultats.

——————- Notes —————————-

1.1 - trouver un format de fichier qui servira de description pour le diag
1.2 - trouver ce qui pourrait servir d'interface utilisateur
2 - trouver le langage de prog
3 - trouver comment représenter le lien logiques entre les entités du système
4 - concevoir un proto diag UML
5 - passage au test / dev
6 - conception des algo après un proto fonctionnel

La création pourrait se faire comme cela :
format de description -> interprétation du format via le prog -> diag SVG

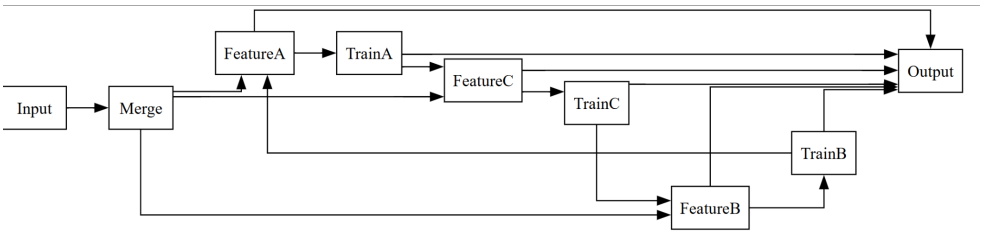
1.1 -> JSON
1.2 -> ?? à voir plus tard pour le moment juste load en ligne de cmd
2 -> Python -> facile au niveau des lib -> moins efficace en temps que du C/Rust
3 -> flèche direct avec les blocs

Test de l'app Graphitz :
- Produit des diagramme verticaux de base
- Peut aussi produire une version pipeline

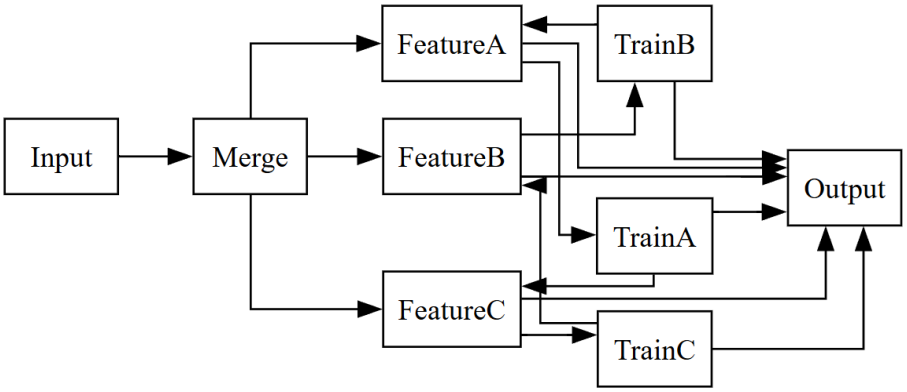
J'ai produit plusieurs tests sur Graphitz et j'ai regardé les optimisations du programme.

J'ai produit plein de versions différentes et j'ai décidé d'en garder 3 :
- je laisse le programme complètement optimiser sans contrainte
- j'impose un minimum de contrainte au programme pour améliorer la visibilité
- j'essaie d'améliorer au maximum le compromis entre lisibilité et optimisation

Voilà le 1er résultat :

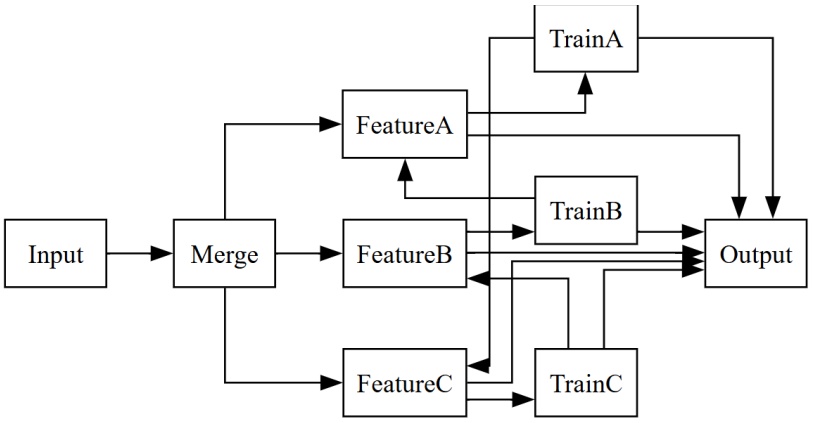


Ce premier résultat résulte d'un .dot assez basique sans aucune contrainte, on voit bien que le programme a clairement optimisé le nombre de coude, les croisements, la distance et pleins d'autres paramètres. Cependant, généralement on sépare par "rank" notre pipeline, on veut pouvoir la visualiser par colonne, c'est bien plus agréable pour l'œil humain même si ce n'est pas optimal mathématiquement.
Voilà donc le 2ème résultat sous contrainte :

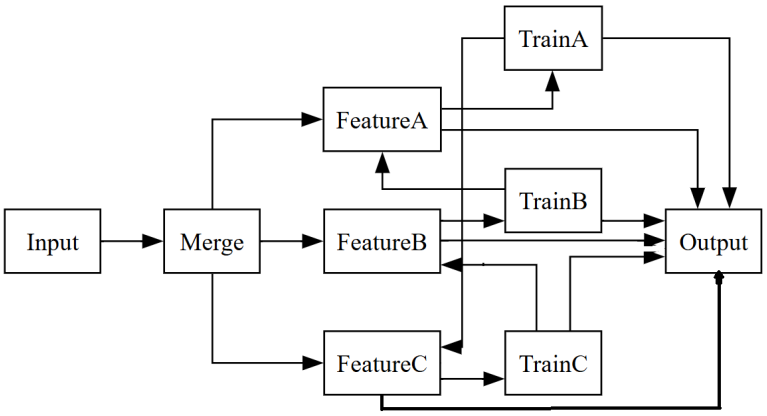


Ici le diagramme, bien que petit, est bien plus compliqué à suivre que le 1er malgré la contrainte de rank (censé rendre plus agréable la pipeline à l'œil). On voit plusieurs problèmes, par exemple trainB est au dessus de trainA pour l'optimisation, cela fausse un peu encore une fois les résultats. On voudrait en fait garder l'ordre des rangs "A,B,C" et garder une bonne optimisation.

Résultat du 3ème test:



Avec ces nouvelles contraintes on voit des problèmes apparaître, et cette fois des problèmes même d'optimisation. On voit par exemple que le FC -> O produit 2 coudes et 1 croisement. C'est clairement suboptimal, et cette fois purement mathématiquement. Pour prouver cela j'ai fais un exemple paint qui produit une version plus optimal FC -> O pour ces deux métriques :



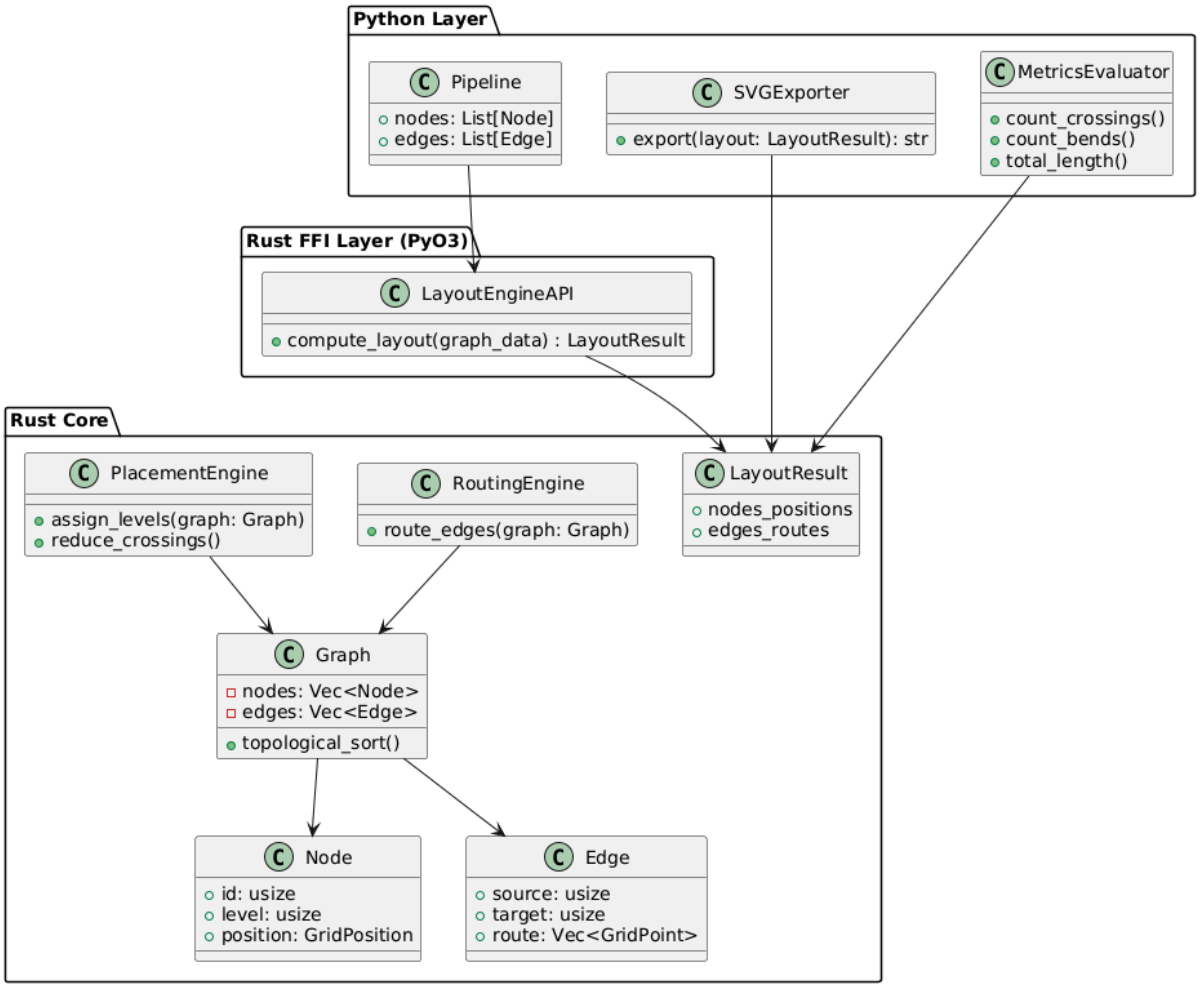
Dans cet exemple on aurait produit 2 coudes et 0 croisements ce qui serait cette fois plus optimal mathématiquement et visuellement. Il y a certainement un moyen de régler ce

problème dans Graphitz en modifiant les paramètres d'optimisations mais cela pourrait nous prendre beaucoup de temps et nous demanderait de lire différentes ressources.

Je vais essayer de combiner Python/Graphitz et Rust.
Graphitz va fortement me servir pour faire l'exportation en SVG et Rust me servira comme langage de base pour créer l'algorithme.

Rust = cœur algorithmique pur (graphes + placement + routing)
Python = orchestration + visualisation + export SVG + évaluation

Plant UML V1 :



—----------USE CASE—----------

aller a src/pipeline_layout

py -m venv .venv

aller a .venv/Srcipts/
→ faire un activate

pip install maturin
pip install customtkinter

-> retour a la racine du projet -> cd ../..
-> aller a python_core -> cd python_core

python build_rust.py

Tout est prêt, maintenant executer : python main.py -h

Pour créer une pipeline lancer : python graph_generator.py
-> cela lance une interface customtkinter qui permet de créer un fichier custom_graph.json qui sera interprété par l'application (cela évite de devoir créer un fichier json à la main)

On peut y ajouter des nœuds via le bouton “ajouter noeud” ou en supprimer EN SURLIGNANT EN BLEU dans la zone le nœud puis en appuyant sur “supprimer noeud”.
Pour les liens il suffit de sélectionner les deux nœuds que l’on veut puis “ajouter lien”, et c’est la même procédure pour supprimer que les nœuds.
A la fin cliquer sur générer JSON et c’est terminé.

exemple de commande simple :
python main.py --file graphs/graph.json --name graph

—-------- Points à remarquer —----------

Il existe un jeu de test déjà à disposition, de “very_simple_graph” à “dropping_node_graph” que j’ai classé par ordre de complexité. Dropping node graph est l’un des DAG les plus dur que j’ai eu à représenter et very simple graph est le premier DAG que j’ai utilisé pour faire mes tests.

Les fichiers utilisés par l’algo sont dans graphs/ (les fichiers json) et les résultats sont dans résultats (les fichiers svg)

Je n’ai pas voulu utiliser graphitz pour faire ce projet car j’avais envie de découvrir par moi même les algos et la génération de svg.

—---------- Points techniques implémentés —----------

“le L routing” -> calcul de 3 pts au lieu de 2 :
-> du centre de la source en X jusqu’au centre du target en X (2 pts) et relié au centre du target en Y (3ème pts)

tri topologique + propagation : on créer une file et 2 hashmaps, une qui servira pour calculer les degrés “entrant” et une qui servira au degré final, on calcul les degrés entrant en fonction des edges (plus il y a de edge plus le degré est haut) et ensuite en les place dans la file en ordre croissant de degré. On défile petit à petit en attribuant le vrai level de chaque en fonction du level de son parent (il prend +1 un degré en fonction du level du parent).

plein de pb arrivent du L routing. j’explore de nombreuses manières de contourner : faire des “couloirs” pour mes edges, verticaux et horizontaux, imbriquer plusieurs “L” etc… Aucun ne marche vraiment bien, je reviens donc à la base : je dois changer la façon dont les nœuds sont placés dans le graphe.

Je reprend le routing horizontal et j’y ajoute un centrage par rapport au layer (je calcul le plus grand layer et je centre mes nœuds à chaque layers en fonction du plus grand layer). J’obtiens de meilleurs résultats mais les edges restent assez mauvais.

Je retrace donc des lignes droites entre les nœuds pour faire les premiers routages, sur les graphes peu denses cette technique très simple marche en fait étonnement très bien… Les problèmes arrivent quand le graphe devient plus dense (plus de nœuds) et quand il y des connexions entre des nœuds de niveaux très différents (plus il y a de saut de niveaux plus c’est complexe).

jexplore donc la technique des “autoroutes” (highways between nodes), quand il y a un saut de plusieurs niveaux d’un coup entre deux noeuds je fais passer le segments par une ligne qui va couper horizontalement ou verticalement plusieurs niveaux mais de façon à ne jamais pouvoir passer derrière un noeud. Cela amène beaucoup de problèmes, plus de coudes, quelques fois plus de croisements et c’est impossible de suivre les lignes étant donné qu’il n’y a qu’une seule autoroute par niveau (à cause du grid layer).

Je reprend avec des lignes droites, j’ajoute à ma structure de segment le fait qu’il y ait un croisement ou non et j’ajoute dans le processing python une couleur aléatoire si il y a effectivement un crossing ou non. Cela est un peu plus compréhensible et facile à suivre mais il reste le problème majeur du segments qui traversent plusieurs niveaux.

Je tente une première approche naïve : je calcule la taille des “layers” en dessous (la taille de la ligne de node) et je fais passer mon segment à l'extérieur. En plus du fait que cela soit assez difficile à implémenter (et que la complexité est vraiment mauvaise) cela créer pleins de problèmes visuels (des segments qui sortent du cadre de l’image, des lignes très grandes difficiles à suivre et qui passent en plus quelques fois quand même derrière les nœuds).

Après plusieurs recherches, je trouve une stratégie très utilisée les “dummy nodes”. La technique revient à créer des faux nœuds que l’on intègre au graphe et l’on fait passer les

segments coupants plusieurs niveaux d'un coup au travers de nœuds qui ne seront pas affichés. Je décide donc de modifier la structure de noeud et d'intégrer. le rendu visuel est meilleur, cependant un problème assez majeur est que tous les segments changent de couleurs quand ils passent par des dummy nodes… Il faut donc que je modifie ma structure d'edge pour y intégrer le nœud source et enfin pouvoir avoir un rendu visuel plutôt satisfaisant. (graph_dummy_stp.svg).

Le résultat permet d'éliminer tout simplement les coudes (mais les segments sont juste un peu cassés à cause des dummy). Cependant cela se fait au détriment des croisements qui sont eux un peu plus nombreux d'environ 20% de plus pour mes tests.

La première idée qui me vient à l'esprit est de diminuer la taille des dummy pour compresser le graphe et le rendre encore plus lisible (ne pas laisser des espaces vides trop grands) mais cette idée s'avère être infructueuse. Le seul graph difficilement lisible à la suite de cela s'avère être dropping_node et c'est clairement le cas le plus extrême que j'ai pu trouver. Malgré cela le graph reste lisible et compréhensible, je décide donc d'arrêter là.